SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1.A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

1. SOAP-Based Web Service for Hospital Management System

Introduction

A hospital management system can use a **SOAP (Simple Object Access Protocol) web service** to exchange patient information securely between departments such as reception, pharmacy, laboratory, billing, and doctors. SOAP

uses XML messages, making communication structured and interoperable across different platforms.

SOAP Message Structure

A SOAP message consists of three main parts:

1. **Envelope** – The root element that identifies the message as a SOAP message.
2. **Header** – Contains optional information such as authentication, authorization, security tokens, and transaction details.
3. **Body** – Contains the actual request or response data.

SOAP Request Example

```
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
            xmlns:h="http://hospital.com/service">
  <soap:Header>
    <h:Authentication>
      <h:Username>doctor01</h:Username>
      <h:Token>ABC123</h:Token>
    </h:Authentication>
  </soap:Header>
```

```
    <soap:Body>
        <h:AddPatient>
            <h:PatientID>P1001</h:PatientID>
            <h:Name>Rahul</h:Name>
            <h:Age>35</h:Age>
            <h:Gender>Male</h:Gender>
            <h:Contact>9876543210</h:Contact>
        </h:AddPatient>
    </soap:Body>
</soap:Envelope>
```

SOAP Response Example

```
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
        <AddPatientResponse>
            <Status>Success</Status>
            <Message>Patient registered successfully</Message>
            <PatientID>P1001</PatientID>
```

```
        </AddPatientResponse>
    </soap:Body>
</soap:Envelope>
```

Main Operations

Operation	Purpose
AddPatient	Registers a new patient
GetPatient	Retrieves patient information
UpdatePatient	Updates patient details
ScheduleAppointment	Creates an appointment
GetAppointment	Retrieves appointment details
GetMedicalRecords	Retrieves medical history
UpdateMedicalRecord	Updates medical information

Data Exchange Process

Client → SOAP Request → Hospital Web Service → Process Request → Database → SOAP Response → Client

For example:

1. A receptionist sends an AddPatient SOAP request.
2. The request contains patient details in XML format.
3. The SOAP server receives and validates the request.
4. The server stores the information in the hospital database.
5. The server generates a SOAP response.
6. The client receives the confirmation.

Why SOAP is Suitable

SOAP is appropriate for a hospital system because:

- It provides **structured XML-based communication**.
- It supports **interoperability** between different programming languages and platforms.
- It supports security through standards such as **WS-Security**.
- It supports reliable communication and standardized error handling using **SOAP Faults**.
- It can support transactions and enterprise-level communication.
- It is suitable for systems where **security, reliability, and strict contracts** are important.

Therefore, SOAP provides a reliable and secure communication mechanism for exchanging sensitive hospital information between departments.

2.An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use **WSDL (Web Services Description Language)** to describe the service.

Design the **WSDL structure** for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

Introduction

WSDL (Web Services Description Language) is an XML-based language used to describe a web service. It tells external applications **what operations are available, what data should be sent, what data will be returned, and where/how the service can be accessed**.

The major components of WSDL are:

1. **Types**
2. **Message**
3. **PortType**
4. **Binding**
5. **Service**

Basic WSDL Structure

```
<definitions
    xmlns="http://schemas.xmlsoap.org/wsdl/"
    xmlns:tns="http://example.com/service"
    targetNamespace="http://example.com/service">

    <types>
        <schema xmlns="http://www.w3.org/2001/XMLSchema">
            <element name="GetDataRequest">
                <complexType>
                    <sequence>
                        <element name="id" type="string"/>
                    </sequence>
                </complexType>
            </element>
        </schema>
    </types>

    <message name="GetDataRequest">
        <part name="parameters"
              element="tns:GetDataRequest"/>
    </message>

    <message name="GetDataResponse">
        <part name="result" type="string"/>
    </message>

    <portType name="DataServicePortType">
        <operation name="GetData">
```

```
            <input message="tns:GetDataRequest"/>
            <output message="tns:GetDataResponse"/>
        </operation>

        <operation name="UpdateData">
            <input message="tns:UpdateDataRequest"/>
            <output message="tns:UpdateDataResponse"/>
        </operation>
    </portType>

    <binding name="DataServiceBinding"
             type="tns:DataServicePortType">

        <soap:binding
            xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
            transport="http://schemas.xmlsoap.org/soap/http"/>

    </binding>

    <service name="DataService">
        <port name="DataServicePort"
              binding="tns:DataServiceBinding">
            <soap:address
                xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
                location="http://example.com/DataService"/>
        </port>
    </service>

</definitions>
```

Components of WSDL

1. Types

The Types section defines the data types used by the service.

For example:

<element name="id" type="string"/>

It specifies that the ID is a string.

2. Message

The Message section defines the data exchanged between the client and server.

Example:

<element name="id" type="string"/>

It specifies that the ID is a string.

2. Message

The **Message** section defines the data exchanged between the client and server.

Example:

```
<message name="GetDataRequest">
    <part name="parameters" element="tns:GetDataRequest"/>
</message>
```

It describes the input message sent to the service.

3. PortType

PortType defines the **operations provided by the service**.

Example:

```
<portType name="DataServicePortType">
   <operation name="GetData"/>
   <operation name="UpdateData"/>
</portType>
```

It acts like an interface for the web service.

4. Binding

The **Binding** section specifies how the service communicates.

For example, it can specify:

- SOAP protocol
- HTTP transport
- Encoding style
- Communication format

5. Service

The **Service** section specifies the actual endpoint where the service can be accessed.

Example:

<soap:address location="http://example.com/DataService"/>

Service Interaction

The overall process can be represented as:

Client → WSDL → Understand Operations → Create SOAP Request → Service Endpoint → SOAP Response → Client

Importance of WSDL

WSDL is important because:

- It provides a **formal service contract**.
- It allows external systems to understand the service automatically.
- It defines available operations and data formats.
- It specifies communication protocols and endpoints.
- It improves interoperability between different applications.
- Tools can use WSDL to generate client/server code automatically.
- It reduces integration errors.

Thus, WSDL acts as a **blueprint or contract** between a web service provider and its clients.

3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SO

AP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

Introduction

In a SOAP-based system, communication takes place through XML-based SOAP requests and responses. If requests are not processed correctly or responses are delayed or incorrect, the problem may occur in the **SOAP message, namespace, endpoint, server, network, or application logic**.

SOAP Communication Process

The normal communication process is:

Client

↓

Create SOAP Request

↓

Validate XML

↓

Send Request through HTTP

↓

SOAP Endpoint

↓

Server Processes Request

↓

Database / Application

↓

Create SOAP Response

↓

Send Response

↓

Client

Possible Problems

1. Message Formatting Errors

Incorrect XML can prevent the server from processing the request.

Example of incorrect XML:

```
<Patient>
   <Name>Rahul
   <Age>25</Age>
</Patient>
```

The <Name> element is not properly closed.

Correct format:

<Patient>

<Name>Rahul</Name>

<Age>25</Age>

</Patient>

2. Incorrect Namespace

SOAP messages depend heavily on namespaces.

For example:

xmlns:h="http://hospital.com/service"

If the client uses an incorrect namespace, the server may not recognize the requested operation.

3. Incorrect Endpoint

The client may send the request to the wrong URL.

Example:

http://server.com/HospitalService

If the actual service is available at:

http://server.com/HospitalService/Patient

the request may fail.

4. Incorrect Operation Name

The client might send:

<GetPatientDetails>

while the service expects:

<GetPatient>

This can cause an operation-not-found error.

5. Incorrect Data Types

The service may expect an integer:

<Age>25</Age>

but the client sends:

<Age>twenty-five</Age>

This can cause validation or processing errors.

6. Authentication and Security Problems

Invalid authentication tokens, expired credentials, or missing WS-Security headers can cause the server to reject the request.

7. Network or Server Problems

Delayed responses may be caused by:

- Network latency
- Server overload

- Database delays
- Incorrect timeout settings
- Service being unavailable

Debugging Approach

Step 1: Check the Endpoint

Verify that the client is using the correct service URL.

Client → Correct SOAP Endpoint

Test whether the endpoint is reachable.

Step 2: Validate the SOAP XML

Check:

- XML syntax
- Opening and closing tags
- Required elements
- Data types
- SOAP Envelope structure

Step 3: Verify Namespaces

Compare the namespaces in the SOAP request with those defined in the WSDL.

xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"

Incorrect namespaces should be corrected.

Step 4: Compare Request with WSDL

The SOAP request should match the WSDL regarding:

- Operation names
- Input parameters
- Output parameters
- Data types
- Namespaces
- Binding

Step 5: Check SOAP Headers

Verify authentication, authorization, security tokens, and other required header information.

Step 6: Check SOAP Faults

SOAP provides a standard error mechanism called **SOAP Fault**.

Example:

```
<soap:Fault>
   <faultcode>soap:Client</faultcode>
   <faultstring>Invalid Patient ID</faultstring>
```

</soap:Fault>

The client can use this information to identify the cause of the error.

Step 7: Check Server Logs

Server logs should be examined for:

- Invalid requests
- Authentication failures
- Database errors
- Application exceptions
- Timeout errors

Step 8: Test Using SOAP Testing Tools

Tools such as SOAP clients can be used to send requests manually and inspect the exact request and response. This helps identify whether the problem is on the client side or server side.

Step 9: Check Timeout and Network Issues

If responses are delayed, check:

- Network connectivity
- Server response time
- Database performance
- HTTP timeout settings

- Server resource usage

How the Corrected System Ensures Reliability

After debugging:

1. The SOAP request follows the correct XML structure.
2. Namespaces match the WSDL.
3. The correct endpoint is used.
4. Input data is properly validated.
5. Authentication is checked.
6. SOAP Faults provide meaningful error information.
7. Server logs help identify failures.
8. Timeout and retry mechanisms can handle temporary failures.

Therefore, proper **message validation, namespace checking, WSDL verification, endpoint testing, error handling, logging, and timeout management** ensure reliable SOAP communication between the client and server.

Code of Conduct:

I _P.Harika(192311243_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Harika

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation